

CANDIDAT	DATE	DURÉE	NOTE
		15 minutes	/ 20

LA CONSIGNE

Une seule bonne réponse par question. Coche, puis compte tes points : **le corrigé est en dernière page**.

Sémantique de référence : SQL standard, vérifiée sur PostgreSQL. Les nuances par moteur (SQL Server, BigQuery, MySQL 8) sont précisées dans le corrigé quand elles divergent.

20 questions

3 niveaux

1 pt / bonne réponse

★ = piège classique

I FONDAMENTAUX · OVER, PARTITION BY, VS GROUP BY

questions 1 à 7 ●●●

Q1 Que renvoie une fonction de fenêtrage (OVER) par rapport à un GROUP BY ?

- ☐ Elle réduit le résultat à une ligne par groupe, comme GROUP BY.
- ☐ Elle conserve toutes les lignes de détail et ajoute la valeur calculée à chacune.
- ☐ Elle supprime les doublons de la table.
- ☐ Elle ne fonctionne que sur une seule colonne à la fois.

Q2 À quoi sert la clause PARTITION BY dans un OVER (...) ?

- ☐ Elle trie le résultat final de la requête.
- ☐ Elle découpe les lignes en sous-groupes indépendants où la fonction est recalculée.
- ☐ Elle physiquement partitionne la table sur le disque.
- ☐ Elle limite le nombre de lignes retournées.

Q3 Quelle écriture donne le total du chiffre d'affaires par région sur chaque ligne de détail ?

- ☐ SUM(ca) GROUP BY region
- ☐ SUM(ca) OVER (PARTITION BY region)
- ☐ SUM(ca) OVER (ORDER BY region)
- ☐ SUM(DISTINCT ca)

Q4 Un OVER () totalement vide (sans PARTITION BY ni ORDER BY) sur SUM(ca) calcule... ★

- ☐ Une erreur de syntaxe.
- ☐ Le total global de toutes les lignes, répété sur chaque ligne.
- ☐ La valeur de la première ligne uniquement.
- ☐ Un cumul progressif ligne par ligne.

Q5 Peut-on combiner GROUP BY et une fonction de fenêtrage dans la même requête ?

- ☐ Non, c'est interdit par le standard SQL.
- ☐ Oui : la fonction de fenêtrage s'applique **après** l'agrégation, sur les lignes déjà regroupées.
- ☐ Oui, mais seulement avec COUNT.
- ☐ Oui, et la fenêtre voit les lignes de détail avant le GROUP BY.

Q6 Que fait COUNT(*) OVER (PARTITION BY client) ?

- ☐ Compte le nombre total de lignes de la table.
- ☐ Compte le nombre de lignes par client, valeur reportée sur chaque ligne du client.
- ☐ Compte les clients distincts.
- ☐ Ne fonctionne pas : COUNT(*) est interdit en fenêtrage.

Q7 Quel est l'avantage clé d'un agrégat fenêtré face à un agrégat avec GROUP BY puis jointure ?

- ☐ Il est toujours plus lent.
- ☐ Il évite une auto-jointure : détail et agrégat coexistent en une seule passe.
- ☐ Il supprime le besoin d'un index.
- ☐ Il ne gère pas les NULL.

II RANKING & NAVIGATION · RANK, LAG, CUMULS

questions 8 à 14 ●●●

Q8 Sur des valeurs à égalité (ex. deux 2^{es} ex æquo), quelle fonction produit la suite 1, 1, 3 ? ★

- ☐ ROW_NUMBER()
- ☐ RANK()
- ☐ DENSE_RANK()
- ☐ NTILE(3)

Q9 Et quelle fonction produit la suite 1, 1, 2 sur la même égalité ?

- ☐ ROW_NUMBER()
- ☐ RANK()
- ☐ DENSE_RANK()
- ☐ PERCENT_RANK()

Q10 Quelle est la particularité de ROW_NUMBER () sur des lignes ex æquo ?

- ☐ Il attribue le même numéro aux ex æquo.
- ☐ Il les départage arbitrairement : le résultat est non déterministe si l'ORDER BY n'est pas unique.
- ☐ Il renvoie NULL en cas d'égalité.
- ☐ Il saute les numéros comme RANK.

Q11 Que renvoie LAG (ca) sans préciser l'offset ?

- ☐ La valeur de ca deux lignes plus haut.
- ☐ La valeur de ca de la ligne précédente (offset 1 par défaut).
- ☐ La valeur de la ligne suivante.
- ☐ La moyenne des lignes précédentes.

Q12 Dans LAG (ca, 1, 0), à quoi sert le troisième argument ?

- ☐ C'est le numéro de la partition.
- ☐ C'est la valeur par défaut renvoyée quand il n'existe pas de ligne précédente.
- ☐ C'est le pas de l'ORDER BY.
- ☐ C'est ignoré par le moteur.

Q13 Pour calculer la variation de CA d'un mois sur l'autre, quelle expression est correcte ?

- ☐ ca - LEAD(ca) OVER (ORDER BY mois)
- ☐ ca - LAG(ca) OVER (ORDER BY mois)
- ☐ ca - SUM(ca) OVER ()
- ☐ ca - RANK() OVER (ORDER BY mois)

Q14 Comment écrire un cumul progressif (running total) du CA dans l'ordre des dates ?

- ☐ SUM(ca) OVER (PARTITION BY date)
- ☐ SUM(ca) OVER (ORDER BY date)
- ☐ SUM(ca) OVER ()
- ☐ RANK() OVER (ORDER BY date)

III NIVEAU SENIOR · FRAMES ROWS/RANGE & ORDRE D'ÉVALUATION

questions 15 à 20 ●●●

Q15 Quel est le cadre (frame) par défaut quand on met un ORDER BY dans le OVER sans clause ROWS/RANGE ? ★

- ☐ ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- ☐ RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- ☐ Tout le partition (UNBOUNDED des deux côtés).
- ☐ La ligne courante seule.

Q16 Quelle est la différence essentielle entre ROWS et RANGE ? ★

- ☐ Aucune, ce sont des synonymes.
- ☐ ROWS compte un nombre **physique** de lignes ; RANGE raisonne en **plage de valeurs** et traite les ex æquo (peers) ensemble.
- ☐ RANGE est plus rapide mais moins précis.
- ☐ ROWS ne marche qu'avec SUM.

Q17 Avec des dates répétées et le cadre RANGE par défaut, ton cumul « bondit » sur les doublons au lieu d'avancer ligne par ligne. Comment forcer un vrai cumul par ligne ?

- ☐ Ajouter DISTINCT.
- ☐ Préciser ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- ☐ Remplacer SUM par COUNT.
- ☐ Supprimer l'ORDER BY.

Q18 Pourquoi LAST_VALUE(x) OVER (ORDER BY id) renvoie souvent la ligne courante au lieu de la dernière du groupe ? ★

- ☐ C'est un bug du moteur.
- ☐ Le cadre par défaut s'arrête à la ligne courante ; il faut ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.
- ☐ LAST_VALUE ignore l'ORDER BY.
- ☐ Il faut un PARTITION BY obligatoire.

Q19 Dans l'ordre logique d'évaluation, à quel moment les fonctions de fenêtrage sont-elles calculées ? ★

- ☐ Avant le WHERE.
- ☐ Après WHERE, GROUP BY et HAVING, mais avant ORDER BY et LIMIT.
- ☐ Tout en dernier, après LIMIT.
- ☐ En même temps que GROUP BY.

Q20 Conséquence directe : peux-tu filtrer dans le WHERE sur le résultat d'un ROW_NUMBER () ?

- ☐ Oui, directement dans le WHERE.
- ☐ Non : il faut l'envelopper dans une CTE ou sous-requête, puis filtrer à l'étage du dessus.
- ☐ Oui, mais seulement avec HAVING.
- ☐ Non, c'est totalement impossible en SQL.

- ✓ **Q1 : Elle conserve toutes les lignes de détail.** C'est LA différence avec GROUP BY : la fenêtre ajoute la valeur calculée à chaque ligne sans collapser le résultat.
- ✓ **Q2 : Elle découpe en sous-groupes indépendants.** La fonction redémarre à zéro sur chaque partition. Rien à voir avec le partitionnement physique de la table.
- ✓ **Q3 : SUM(ca) OVER (PARTITION BY region).** Sans ORDER BY, c'est le total de la région, reporté sur chaque ligne. Avec ORDER BY, on obtiendrait un cumul (autre résultat).
- ✓ **Q4 : Le total global, répété sur chaque ligne.** Un OVER () vide fait de toute la table une seule fenêtre, sans cumul (pas d'ORDER BY).
- ✓ **Q5 : Oui, la fenêtre s'applique après l'agrégation.** L'ordre logique place le fenêtrage après GROUP BY/HAVING : la fenêtre voit les lignes déjà regroupées, jamais le détail brut.
- ✓ **Q6 : Le nombre de lignes par client, reporté partout.** Pratique pour garder le détail tout en affichant « X commandes pour ce client » sur chaque ligne.
- ✓ **Q7 : Il évite une auto-jointure.** Détail et agrégat en une passe, là où GROUP BY + jointure impose deux lectures et un raccord.
- ✓ **Q8 : RANK () → 1, 1, 3.** Il attribue le même rang aux ex æquo puis saute les rangs intermédiaires (pas de 2).
- ✓ **Q9 : DENSE_RANK () → 1, 1, 2.** Même rang aux ex æquo, mais sans trou : le rang suivant est 2.
- ✓ **Q10 : Il départage arbitrairement (non déterministe).** ROW_NUMBER () donne toujours des numéros uniques ; si l'ORDER BY n'est pas unique, l'ordre des ex æquo n'est pas garanti. Ajoute un tie-breaker (ex. id).
- ✓ **Q11 : La ligne précédente (offset 1).** LAG (ca) = LAG (ca, 1). LEAD regarde au contraire vers l'avant.
- ✓ **Q12 : La valeur par défaut.** Le 3^e argument remplace le NULL renvoyé en bord de partition (ici 0). Utile pour éviter un NULL dans une soustraction.
- ✓ **Q13 : ca - LAG(ca) OVER (ORDER BY mois).** On compare au mois précédent. LEAD donnerait l'écart avec le mois suivant (signe inversé).

- ✓ **Q14 : SUM(ca) OVER (ORDER BY date).** L'ORDER BY dans le OVER transforme l'agrégat en cumul progressif (du début jusqu'à la ligne courante).
- ✓ **Q15 : RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.** Le piège senior n°1 : le défaut est RANGE, jamais ROWS. Confirmé par la doc PostgreSQL.
- ✓ **Q16 : ROWS = lignes physiques, RANGE = plage de valeurs (peers groupés).** Avec RANGE, toutes les lignes de même valeur d'ORDER BY sont traitées comme une seule frontière de cadre.
- ✓ **Q17 : Préciser ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.** Conséquence directe de Q15/Q16 : avec RANGE, les dates en doublon partagent le même cumul ; ROWS avance ligne par ligne.
- ✓ **Q18 : Le cadre par défaut s'arrête à la ligne courante.** Pour la vraie dernière valeur du groupe, étends le cadre : ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING. Même piège pour NTH_VALUE.
- ✓ **Q19 : Après WHERE/GROUP BY/HAVING, avant ORDER BY/LIMIT.** D'où le fait qu'une fenêtre voit les lignes déjà filtrées et regroupées.
- ✓ **Q20 : Non, il faut une CTE ou sous-requête.** Le fenêtrage étant évalué après le WHERE, on ne peut pas y référencer un ROW_NUMBER () ; on le calcule dans une CTE puis on filtre au-dessus. (On peut en revanche l'utiliser dans l'ORDER BY.)

TA NOTE SUR 20

0-11

Niveau junior : maîtrise d'abord OVER, PARTITION BY et la différence avec GROUP BY ; le ranking suivra vite.

12-16

Profil opérationnel. Creuse les frames ROWS/RANGE et l'ordre d'évaluation pour passer un cap.

17-20

Niveau senior : les fenêtres ne te piègent plus, même sur le cadre par défaut et LAST_VALUE.

→ Poste ta note en commentaire, sans tricher.